

Tu recorrido

Módulo 1: Configuración del entorno (25 minutos)

Primero, preparaste tu entorno de desarrollo: instalaste Python, creaste un entorno virtual, instalaste el ADK y configuraste claves de API. Aprendiste la secuencia de configuración correcta y verificaste que todo funcionara con la creación y ejecución de tu primer agente con `adk create` y `adk web`.

Conceptos clave dominados:

- Instalación del ADK con `pip install google-adk`
- Creación de entornos virtuales para proyectos de Python
- Configuración de archivos `.env` con `GOOGLE_API_KEY` después de ejecutar `adk create`
- Comprensión de la estructura de archivos generada: `agent.py`, `__init__.py` y `.env`
- Verificación de la configuración con `adk web`

Módulo 2: Entiende tu primer agente (20 minutos)

Aprendiste los cuatro parámetros principales que definen cada agente: `model`, `name`, `instruction` y `description`. Transformaste al agente predeterminado en un tutor de matemáticas especializado. Para ello, personalizaste sus instrucciones y descubriste la diferencia entre la descripción (para otros agentes) y la instrucción (para el comportamiento de este agente).

Conceptos clave dominados:

- Los cuatro parámetros principales: `model`, `name`, `description` e `instruction`
- La convención de variables de `root_agent` que buscan las herramientas del ADK
- Escribir instrucciones eficaces que definan la personalidad y el comportamiento del agente
- Comprender cuándo usar `description` en lugar de `instruction`
- Prueba de agentes con `adk web`

Módulo 3: Otras formas de ejecutar tu agente (15 minutos)

Aprendiste tres formas diferentes de ejecutar agentes además de la interfaz web: la ejecución de terminal con `adk run`, la implementación de la API de REST con `adk api_server` y la ejecución programática en código de Python. Cada método satisface diferentes necesidades, desde pruebas rápidas hasta integración de producción.

Conceptos clave dominados:

- `adk run` para pruebas rápidas basadas en la terminal
- `adk api_server` para implementar agentes como APIs de REST
- La ejecución programática con ejecutor, sesiones y bucles de eventos asíncronos

- Cuándo usar cada método de ejecución
- Ejecución desde el directorio del agente en lugar del directorio superior

Módulo 4: Configuración del agente con YAML (15 minutos)

Exploraste una segunda forma de definir agentes con archivos de configuración YAML en lugar de código de Python. Aprendiste que `adk create --type=config` genera un archivo `root_agent.yaml`, lo que hace que la creación de agentes sea accesible para personas que no son programadores. Descubriste cuándo usar YAML (agentes simples y creación rápida de prototipos) en lugar de código de Python (sistemas complejos de múltiples agentes y herramientas personalizadas).

Conceptos clave dominados:

- Crear agentes basados en YAML con `adk create --type=config`
- Escribir los parámetros de configuración del agente en formato YAML
- Comprender que la configuración del agente YAML es solo para Python
- Saber cuándo usar YAML en lugar de Python para la definición del agente
- Ejecutar agentes YAML de la misma manera que los agentes de Python

Qué puedes hacer ahora

Con las habilidades de este curso, podrás realizar las siguientes tareas:

1. **Crear agentes personalizados** para cualquier propósito: tutores, asistentes o escritores creativos
2. **Ejecutar agentes de diversas formas** según tus necesidades: IU web, terminal, API o de forma programática
3. **Definir agentes en Python o YAML** según tus preferencias y caso de uso
4. **Crear aplicaciones impulsadas por agentes** utilizando el patrón Runner con sesiones

Conclusiones principales para recordar

A medida que avances, ten en cuenta estas afirmaciones:

1. **Cuatro parámetros principales.** Cada agente necesita un modelo (model), un nombre (name), una descripción (description) y una instrucción (instruction).
2. **El parámetro instruction es fundamental.** Este parámetro define cómo se comporta tu agente y cómo interactúa.
3. **Hay diversos métodos de ejecución:** Elige `adk web` para el desarrollo, `adk run` para pruebas rápidas, `adk api_server` para APIs o ejecución programática para apps personalizadas.
4. **Python o YAML:** Usa YAML para mayor simplicidad y accesibilidad, y Python para sistemas complejos.
5. **La configuración del entorno es importante.** Siempre configura `.env` con

GOOGLE_API_KEY después de `adk create`.

Comunidad y asistencia

Referencias principales

-  [Documentación del ADK](#) – Referencia oficial del ADK
-  [Guía de inicio rápido del ADK de Python](#): Guía de introducción
-  [Documentación de configuración del agente](#): Referencia de configuración de YAML

Recursos

De Google

- [Codelab – Creación de agentes de IA con el ADK \(conceptos básicos\)](#): Un codelab práctico para desarrollar habilidades básicas de creación de agentes

De la comunidad

- [Video: Instructivo del Kit de desarrollo de agentes de Google Cloud | Crea tu primer agente del ADK](#): Un instructivo para principiantes sobre cómo crear tus propios agentes
- [Video: Capacitación de la comunidad de IA de Google: Introducción al Kit de desarrollo de agentes de Google](#), una capacitación completa sobre el ADK de Google
- [Artículo - Capítulo 1: Introducción a los agentes de IA y al Kit de desarrollo de agentes \(ADK\)](#), un análisis profundo sobre la creación de agentes de IA
- [Curso - IA práctica: Creación de agentes con el Kit de herramientas de desarrollo de agentes de Google \(ADK\)](#), un curso de LinkedIn sobre la creación práctica de agentes de IA
- [Crea tu primer agente de IA con el ADK: El Kit de desarrollo de agentes de Google](#), un instructivo paso a paso que incluye la configuración del entorno y la creación de tu primer agente conversacional
- [Tu primer agente con el ADK: Crea un administrador de tareas pendientes de Google Tasks](#), un ejemplo práctico de cómo crear un agente del mundo real con autenticación de OAuth
- [Creación e implementación de un agente de Python del ADK desde Google Cloud Shell](#), un artículo que explora la configuración de un entorno alternativo con Cloud Shell en lugar de la instalación local
- [Usa tu agente de IA del ADK en una IU](#): Cómo integrar tu agente en una IU web con ejemplos de código prácticos
- [Creación e implementación de agentes de IA en minutos con el ADK de Google \(parte 1\)](#): Un instructivo completo desde la configuración hasta la implementación, que incluye múltiples métodos de ejecución

¿Tienes preguntas? Publícalas en el [foro de la](#)

comunidad

Tarjeta de referencia rápida

Comandos esenciales

```
# Configuración del entorno
python3 -m venv adk-env          # Crea un entorno virtual
source adk-env/bin/activate      # Activa (macOS/Linux)
adk-env\Scripts\activate        # Activa (Windows)
pip install google-adk          # Instala el ADK

# Crea agentes
adk create my_agent              # Crea un agente basado en Python
adk create --type=config my_agent # Crea un agente basado en YAML

# Ejecuta agentes
adk web                          # Interfaz web (desde el directorio del agente)
)
adk web my_agent                 # Interfaz web (desde el directorio superior )
adk run                          # Ejecuta la terminal
adk api_server                   # Servidor de la API de REST
```

Patrón de agentes basados en Python

```
from google.adk.agents.llm_agent import Agent

root_agent = Agent(
    model='gemini-2.5-flash',
    name='math_tutor_agent',
    description='Ayuda a los estudiantes a aprender álgebra guiándolos a través de los pasos para la resolución de problemas.',
    instruction="""Eres un tutor de álgebra paciente y motivador.

    Este es tu enfoque de enseñanza:
    1. Cuando un estudiante pregunta, primero debes entender cuál es la dificultad
    2. Divide el problema en pasos más pequeños y manejables
    3. Guíalo para que pueda descubrir la respuesta, en lugar de dársela directamente
    4. Ofrece refuerzos positivos por su esfuerzo y progreso
    5. Usa un lenguaje sencillo y evita la jerga

    Siempre mantén un tono comprensivo y paciente."""
)
```

Patrón de agentes basados en YAML

```
# root_agent.yaml
name: math_tutor_agent
```

```
model: gemini-2.5-flash
description: Ayuda a los estudiantes a aprender álgebra guiándolos a través de pasos de resolución de problemas.
instruction: |
```

```
    Eres un tutor de álgebra paciente y motivador.
```

```
Este es tu enfoque de enseñanza:
```

1. Cuando un estudiante pregunta, primero debes entender cuál es la dificultad
2. Divide el problema en pasos más pequeños y manejables
3. Guíalo para que pueda descubrir la respuesta, en lugar de dársela directamente
4. Ofrece refuerzos positivos por su esfuerzo y progreso
5. Usa un lenguaje sencillo y evita la jerga

```
Siempre mantén un tono comprensivo y paciente.
```

Patrón de ejecución programática

```
import asyncio
from google.adk.agents.llm_agent import Agent
from google.adk.runners import Runner
from google.adk.sessions import InMemorySessionService
from google.genai.types import Content, Part

# Crea un agente
agent = Agent(
    model='gemini-2.5-flash',
    name='math_tutor',
    instruction='Eres un tutor de matemáticas paciente.'
)

# Configura el ejecutor
session_service = InMemorySessionService()
runner = Runner(
    agent=agent,
    app_name='my_app',
    session_service=session_service
)

# Ejecuta el agente
async def run_agent():
    session = await session_service.create_session(
        app_name='my_app',
        user_id='user_1',
        session_id='session_1'
    )

    user_message = Content(
        role='user',
        parts=[Part(text='¿Cuánto es  $2x + 5 = 13$ ?')]
    )

    async for event in runner.run_async(
        user_id='user_1',
        session_id='session_1',
```

```
        new_message=user_message
    ):
        if event.is_final_response() and event.content and event.content.parts:
            print(event.content.parts[0].text)

# Ejecuta
asyncio.run(run_agent())
```